

C++ Theory Guide

C++ Basics

Welcome to your first chapter in C++! This guide covers all the core concepts you'll need to complete your practice exercises. Whether you're new to programming or just new to C++, this foundation will help you think like a developer and solve real problems.

Input and Output

Input allows your program to receive information from the user.

Output is how your program displays information back.

- `std::cin` – reads input
- `std::cout` – writes output

Example:

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    cin >> age;
    cout << "You are " << age << " years old.";
    return 0;
}
```

2 Variables and Data Types

A **variable** is a named container for a value.

A **data type** defines what kind of value the variable holds.

- `int` – whole numbers
- `double` – decimal numbers
- `char` – single characters
- `bool` – true or false
- `string` – text (requires `#include <string>`)

```
int age = 20;  
double height = 1.75;  
char grade = 'B';  
bool isStudent = true;  
string name = "Alex";
```

3 Operators: Arithmetic & Logical

12 34 Arithmetic Operators

- `+` Addition
- `-` Subtraction
- `*` Multiplication
- `/` Division
- `%` Modulus (remainder)

⚠ Integer division truncates results. Use `double` if you need decimals.

4 Conditions: if, else, switch

if / else if / else

```
if (score >= 90)
{
    // Grade A
}
else if (score >= 80)
{
    // Grade B
}
else
{
    // Grade C or lower
}
```

This checks the value of `score` and prints a message based on the range it falls into.

Using Logical Operators in Conditions

```
int age = 17;
bool isStudent = true;

if (age < 18 || isStudent)
{
    cout << "You qualify for a discount.";
}
else
{
    cout << "No discount available.";
}
```

This condition uses the `||` operator (logical OR). It checks if either the person is under 18 *or* is a student. If at least one condition is true, the message will show the discount.

Other logical operators you can use in `if` statements:

- `&&` – AND: both conditions must be true
- `||` – OR: at least one condition must be true
- `!` – NOT: reverses a condition (examples : `age != 18` , `!isStudent` (false))

switch

```
switch (grade)
{
    case 'A':
        // Excellent!
```

```

        break;
    case 'B':
        //Good!
        break;
    default:
        // Try again
}

```

Loops

Loops allow you to repeat actions in your code. This is useful when you want to do something multiple times without writing the same code again.

while Loop

```

int guess;
int number = 7;

cout << "Guess the number: ";
cin >> guess;

while (guess != number)
{
    cout << "Wrong! Try again: ";
    cin >> guess;
}

```

The while loop checks the condition **before** each repetition. If the condition is false right away, the loop may never run.

do...while Loop

```

int guess;
int number = 7;

do
{
    cout << "Guess the number: ";
    cin >> guess;
} while (guess != number);

```

The do...while loop always runs **at least once**, because the condition is checked **after** the first iteration. This is useful when you want to ensure the user is prompted at least one time.

for loop

```

for (int i = 1; i <= 10; i++)
{
    // Display from 1 to 10
    cout << i << endl;
}

```

6 Functions in C++

Functions help you organize your code into smaller, manageable pieces. They let you group tasks, avoid repeating code, and make your program easier to read and update.

In C++, a function has **four main parts**:

1. **Return type** – what the function gives back (e.g., `int`, `string`, or `void` if nothing)
2. **Function name** – like a label (e.g., `greet`, `addNumbers`)
3. **Parameters** – input values the function needs to work
4. **Function body** – the actual instructions

1/Functions with *no* return value and *no* parameters

```
void greet()  
{  
    cout << "Hello, world!" << endl;  
}
```

Use this when: You want to print or perform a task that doesn't depend on input and doesn't need to return a result.

2/Functions with *no* return value but *with* parameters

```
void greetUser(string name)  
{  
    cout << "Hello, " << name << "!" << endl;  
}
```

Use this when: You want to pass in data, use it inside the function, but you don't need to return anything.

3/Functions *with* a return value and *no* parameters

```
int getLuckyNumber()  
{  
    return 7;  
}
```

Use this when: You want to generate or calculate something without needing input.

4/ Functions *with* a return value and *with* parameters

```
int add(int a, int b)  
{  
    return a + b;  
}
```

Use this when: You want to process some input and give a result back to the caller.

💡 How to Use a Function

```
int main()
{
    greet(); // calls a void function
    greetUser("Alice"); // passes a string
    int lucky = getLuckyNumber(); // stores return value
    int result = add(5, 3); // stores sum
    cout << "Sum is: " << result << endl;
    return 0;
}
```

🧠 Why Functions Matter

- They help you **reuse** code
- Make your programs **modular** and easy to debug
- Help you **organize logic** into smaller steps

7 Scope: Local vs Global

Scope defines where a variable can be used in your program.

◆ Local Scope

Variables declared inside a function or block exist **only there**. They are *created* when the block runs and *destroyed* when it ends.

```
void greet()
{
    string name = "Sam"; // Local to greet()
    cout << "Hi " << name << endl;
}
```

A local variable **must be initialized** before you use it, or it may contain garbage data.

```
void test()
{
    int x; // uninitialized local variable
    cout << x; // ❌ Unsafe: value is unpredictable
}
```

✅ **Best practice:** Always initialize local variables before use.

◆ Global Scope

A global variable is declared **outside** all functions. It can be accessed or modified from anywhere in the file.

```
int counter = 0; // global and initialized

void increase()
{
    counter++; // modifies the global variable
}
```

Global variables are automatically initialized to **zero** (for numeric types) if not explicitly given a value.

```
int total; // implicitly initialized to 0
```

Why Scope and Initialization Matter

- **Scope** helps manage complexity in large programs.
- Uninitialized **local** variables can cause bugs.
- **Global** variables, while convenient, can lead to unintended side effects if overused.

8 Strings and Characters

To use strings, include the string library:

```
#include <string>
```

Declaring and Using Strings

```
string name = "Alice";
cout << "Hello, " << name << endl;
```

Character Access

Strings are made of characters you can access with indexes.

```
char first = name[0]; // 'A'
```

String Functions

- `length()` – returns the number of characters
- `substr(start, length)` – returns a part of the string
- `find("text")` – finds the position of text

- `toupper(c) / tolower(c)` – change character case

Looping Through a String

```
for (int i = 0; i < name.length(); i++)  
{  
    cout << toupper(name[i]) << endl;  
}
```

9 Arrays

An array holds multiple values of the same type.

```
int scores[3] = {85, 90, 78};
```

- Index starts at 0
- `scores[0]` is 85, `scores[1]` is 90

Accessing and Modifying

```
scores[2] = 100;
```

Looping Through an Array

```
for (int i = 0; i < 3; i++)  
{  
    cout << scores[i] << endl;  
}
```

Array Size

In modern C++, you can also use:

```
int size = sizeof(scores) / sizeof(scores[0]);
```


Structure and pointers

Welcome back! In Chapter 2, we'll go deeper into working with arrays, pointers, dynamic memory, structures in C++. This guide provides the theory and concepts you'll need to master all the exercises in this chapter.

C-style Strings

C-style strings are arrays of characters ending with a null character `'\0'`.

- Access characters using array indexing
- Use `strlen()`, `strcpy()`, `strcat()`, `strcmp()` from `<cstring>`

```
char name[] = "Alice";
```

Working with Arrays

Arrays store fixed-size sequences of elements of the same type.

- Index starts at 0
- Loop through arrays for calculations like total, max, min, average

```
float values[5] = {2.5, 3.0, 4.1, 5.5, 6.3};
```

Pointers and Memory Access

Pointers store memory addresses. Use `*` to dereference and `&` to get an address.

```
double balance = 100.0;
double* ptr = &balance;
cout << *ptr; // outputs 100
```

Use pointers to modify variables in functions or simulate basic memory control.

4 Dynamic Memory

Use `new` to allocate memory at runtime and `delete` to free it.

```
int* data = new int[10];
delete[] data;
```

5 Structures (struct)

`struct` is used to group related variables (members) under one name.

```
struct Employee {
    int id;
    string name;
    double salary;
};
```

6 Pass-by-Value vs. Pass-by-Reference

- **Pass-by-value:** Copies the value (won't change original)
- **Pass-by-reference:** Passes the address (can modify original)

```
void modify(int& x) { x += 10; }
```

7 Arrays of Structures

Combine arrays with structures to manage multiple records efficiently.

```
Student students[3];
```

8 Functions Returning Pointers

You can return pointers from functions, but you must be careful with memory ownership.

```
float* createArray(unsigned int size)
{
    return new float[size];
}
```

9 C-style Strings vs. `std::string`

`char[ ]` (C-style) vs. `string` (modern C++):

- `char name[ 20 ]` – Fixed size, manual management
- `string name` – Dynamic, easier to use

Object-Oriented Programming (Part 1)

Encapsulation: Wrapping Data and Methods

Encapsulation is one of the fundamental principles of Object-Oriented Programming (OOP). It refers to the idea of **bundling** the data (attributes) and the methods (functions) that operate on that data into a **single unit**, called a **class**. Encapsulation helps in restricting access to some of the object's components and prevents external interference.

Private: Members are only accessible within the class itself.

Public: Members are accessible from anywhere in the program.

In C++, encapsulation is achieved through **access modifiers** and **getter/setter methods**.

Code Example:

```
// Vehicle.h
#ifndef VEHICLE_H
#define VEHICLE_H

#include <string>
using namespace std;

class Vehicle {
private:
    string model;
    int speed;

public:
    // Constructor
    Vehicle(string vehicleModel);

    // Getter for speed
    int getSpeed() const;

    // Setter for speed
    void setSpeed(int newSpeed);

    // Method to display vehicle info
    void displayInfo() const;
};

#endif

// Vehicle.cpp
#include "Vehicle.h"
#include <iostream>
```

```

using namespace std;

Vehicle::Vehicle(string vehicleModel) {
    model = vehicleModel;
    speed = 0; // Default speed
}

int Vehicle::getSpeed() const {
    return speed;
}

void Vehicle::setSpeed(int newSpeed) {
    if (newSpeed >= 0) {
        speed = newSpeed;
    } else {
        cout << "Speed cannot be negative!" << endl;
    }
}

void Vehicle::displayInfo() const {
    cout << "Vehicle Model: " << model << ", Speed: " << speed <<
    " km/h" << endl;
}

```

2 Constructors and Destructors: Initialization and Cleanup

In C++, **constructors** are special member functions used to initialize an object when it is created. **Destructors** are used to clean up when the object is destroyed.

- **Constructor:** Initializes an object with default or specific values.
- **Destructor:** Performs necessary cleanup when the object is destroyed.

Code Example:

```

// SmartDevice.h
#ifndef SMARTDEVICE_H
#define SMARTDEVICE_H

#include <string>
using namespace std;

class SmartDevice {
private:
    string name;
    bool isOn;

public:

```

```

    // Constructor to initialize the device
    SmartDevice(string deviceName);

    // Destructor to display device cleanup
    ~SmartDevice();

    // Method to turn the device on
    void turnOn();

    // Method to display device status
    void displayStatus() const;
};

#endif

// SmartDevice.cpp
#include "SmartDevice.h"
#include <iostream>
using namespace std;

SmartDevice::SmartDevice(string deviceName) {
    name = deviceName;
    isOn = false;
    cout << name << " device created." << endl;
}

SmartDevice::~SmartDevice() {
    cout << name << " device destroyed." << endl;
}

void SmartDevice::turnOn() {
    isOn = true;
    cout << name << " is now ON." << endl;
}

void SmartDevice::displayStatus() const {
    cout << "Device: " << name << ", Status: " << (isOn ? "ON" :
"OFF") << endl;
}

```

3 Access Modifiers: Controlling Access to Data

Access modifiers define the accessibility of class members:

- **Private:** Members are accessible only within the class.
- **Public:** Members are accessible from outside the class.
- **Protected:** Members are accessible in the class and derived classes.

Code Example:

```
// HomeDevice.h
#ifndef HOMEDEVICE_H
#define HOMEDEVICE_H

#include
using namespace std;

class HomeDevice {
private:
    string deviceName;
    bool status;

public:
    // Constructor
    HomeDevice(string name);

    // Getter for status
    bool getStatus() const;

    // Setter to turn on the device
    void turnOn();

    // Method to display device status
    void displayStatus() const;
};

#endif

// HomeDevice.cpp
#include "HomeDevice.h"
#include
using namespace std;

HomeDevice::HomeDevice(string name) {
    deviceName = name;
    status = false; // Device is off initially
```

```

}

bool HomeDevice::getStatus() const {
    return status;
}

void HomeDevice::turnOn() {
    status = true;
    cout << deviceName << " is now ON." << endl;
}

void HomeDevice::displayStatus() const {
    cout << "Device: " << deviceName << ", Status: " << (status ?
"ON" : "OFF") << endl;
}

```

4 Getters and Setters: Accessing and Modifying Data

Getters and **setters** are methods used to retrieve and modify **private** class members. They provide a controlled interface for accessing or changing the internal state of an object.

Code Example:

```

// SmartLamp.h
#ifndef SMARTLAMP_H
#define SMARTLAMP_H

#include <string>
using namespace std;

class SmartLamp {
private:
    string model;
    int brightness;

public:
    // Constructor to initialize the lamp
    SmartLamp(string lampModel);

    // Getter for brightness
    int getBrightness() const;

    // Setter for brightness
    void setBrightness(int level);

    // Method to display lamp status
    void displayStatus() const;
};

#endif

```



```

// SmartLamp.cpp
#include "SmartLamp.h"
#include
using namespace std;

SmartLamp::SmartLamp(string lampModel) {
    model = lampModel;
    brightness = 0; // Default brightness level is 0
}

int SmartLamp::getBrightness() const {
    return brightness;
}

void SmartLamp::setBrightness(int level) {
    if (level >= 0 && level <= 100) {
        brightness = level;
    } else {
        cout << "Invalid brightness level!" << endl;
    }
}

void SmartLamp::displayStatus() const {
    cout << "Lamp Model: " << model << ", Brightness: " <<
brightness << "%" << endl;
}

```

5 Method Overloading: Multiple Methods with the Same Name

Method overloading allows you to define multiple methods with the same name, but different parameters. This enables you to perform similar operations with different types of arguments.

Code Example:

```

// Calculator.h
#ifndef CALCULATOR_H
#define CALCULATOR_H

class Calculator {
public:
    // Overloaded method to add two integers
    int add(int a, int b);

    // Overloaded method to add three integers
    int add(int a, int b, int c);

    // Method to add two floating-point numbers
    float add(float a, float b);
}

```

```
};

#endif

// Calculator.cpp
#include "Calculator.h"

int Calculator::add(int a, int b) {
    return a + b;
}

int Calculator::add(int a, int b, int c) {
    return a + b + c;
}

float Calculator::add(float a, float b) {
    return a + b;
}
```

6 The `this` Pointer: Referring to the Current Object

The **this** pointer is an implicit pointer passed to all non-static member functions. It refers to the **current instance** of the class, enabling you to access the object that invoked the method.

Code Example:

```
// SmartLight.h
#ifndef SMARTLIGHT_H
#define SMARTLIGHT_H

#include
using namespace std;

class SmartLight {
private:
    string model;
    bool isOn;

public:
    // Constructor
    SmartLight(string lightModel);

    // Method to turn on the light using the 'this' pointer
    void turnOn();

    // Method to display the status
    void displayStatus() const;
};
```

```
#endif
```

```
// SmartLight.cpp
#include "SmartLight.h"
#include
using namespace std;

SmartLight::SmartLight(string lightModel) {
    model = lightModel;
    isOn = false;
}

void SmartLight::turnOn() {
    this->isOn = true; // Using 'this' pointer to refer to the
current object
    cout << model << " light is now ON." << endl;
}

void SmartLight::displayStatus() const {
    cout << "Smart Light Model: " << model << ", Status: " <<
(isOn ? "ON" : "OFF") << endl;
}
```



Summary of Key Concepts

- **Encapsulation:** Protects the data by hiding internal variables and providing controlled access through methods.
- **Constructors and Destructors:** Used to initialize objects and clean up resources when they are no longer needed.
- **Access Modifiers:** Control the visibility of class members (private, public, protected).
- **Getters and Setters:** Provide a controlled interface for reading and modifying private data members.
- **Method Overloading:** Enables defining multiple methods with the same name but different parameter lists.
- **The `this` Pointer:** Refers to the current instance of the class, useful for resolving naming conflicts and returning the current object.
- **The `const` Keyword:** Ensures immutability, restricting modification of data or method behavior.

Object-Oriented Programming (Part 2)

Inheritance: Reusing Code with Derived Classes

Inheritance allows a class (the derived class) to inherit properties and behaviors (methods) from another class (the base class). It enables code reuse and can help establish a relationship between the base and derived classes.

In this example, we model a **Shape** class as the base class and two derived classes, **Circle** and **Rectangle**, to represent specific types of shapes. These derived classes inherit from **Shape** and have their own specific methods that are not shared with the other class.

Code Example:

Shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

#include <string>
using namespace std;

class Shape {
protected:
    string color;

public:
    Shape(string c); // Constructor

    // Display basic shape info (shared functionality)
    void displayColor() const;

    // Virtual destructor for proper cleanup
    virtual ~Shape();
};
```

#endif Shape.cpp

```
#include "Shape.h"
#include <iostream>

Shape::Shape(string c) : color(c) {}

void Shape::displayColor() const {
    cout << "Shape color: " << color << endl;
}

Shape::~~Shape() {}
```

Circle.h

```

#ifndef CIRCLE_H
#define CIRCLE_H

#include "Shape.h"
#include <string>
using namespace std;

class Circle : public Shape {
private:
    double radius;

public:
    Circle(string c, double r); // Constructor

    // Method specific to Circle, not shared with Rectangle
    void calculateArea() const;

    // Display specific Circle info
    void displayCircleDetails() const;
};

```

#endif
Circle.cpp

```

#include "Circle.h"
#include <iostream>

Circle::Circle(string c, double r) : Shape(c), radius(r) {}

void Circle::calculateArea() const {
    double area = 3.14 * radius * radius;
    cout << "Area of the circle: " << area << endl;
}

void Circle::displayCircleDetails() const {
    displayColor();
    cout << "Circle radius: " << radius << endl;
}

```

Rectangle.h

```

#ifndef RECTANGLE_H
#define RECTANGLE_H

#include "Shape.h"

class Rectangle : public Shape {
private:
    double length, width;

public:
    Rectangle(string c, double l, double w); // Constructor

```

```

    // Method specific to Rectangle, not shared with Circle
    void calculatePerimeter() const;

    // Display specific Rectangle info
    void displayRectangleDetails() const;
};

#endif
Rectangle.cpp

#include "Rectangle.h"

Rectangle::Rectangle(string c, double l, double w) : Shape(c),
length(l), width(w) {}

void Rectangle::calculatePerimeter() const {
    double perimeter = 2 * (length + width);
    cout << "Perimeter of the rectangle: " << perimeter << endl;
}

void Rectangle::displayRectangleDetails() const {
    displayColor();
    cout << "Rectangle dimensions: " << length << " x " << width
<< endl;
}

```

Key Differences:

- **Shared Method:** Shape class has a `displayColor` method that is common to both Circle and Rectangle, since they both have a color attribute.
- **Unique Methods:**
 - Circle has the method `calculateArea` to calculate the area of the circle.
 - Rectangle has the method `calculatePerimeter` to calculate the perimeter of the rectangle.
- Both classes also have their own methods for displaying specific details of the shape (i.e., `displayCircleDetails` and `displayRectangleDetails`).

Example Usage:

Main.cpp

```

#include "Circle.h"
#include "Rectangle.h"

```

```

int main() {
    Shape* shapes[2];

    // Create objects of Circle and Rectangle
    shapes[0] = new Circle("Red", 5);
    shapes[1] = new Rectangle("Blue", 4, 6);

    // Use the display and specific methods for each shape
    dynamic_cast(shapes[0])->displayCircleDetails(); // Circle-
specific details
    dynamic_cast(shapes[1])->displayRectangleDetails(); //
Rectangle-specific details

    // Call specific methods for calculations
    dynamic_cast(shapes[0])->calculateArea(); // Area of circle
    dynamic_cast(shapes[1])->calculatePerimeter(); // Perimeter
of rectangle

    // Cleanup
    delete shapes[0];
    delete shapes[1];
}

```

2 Polymorphism: Different Implementations with a Common Interface

Polymorphism allows a base class pointer or reference to call methods from derived classes. It ensures that the correct method is executed based on the actual object type, rather than the pointer type.

In this case, we use polymorphism with our **Vehicle** class to handle different types of vehicles (like **Car** and **Truck**) through a common interface.

Code Example:

Vehicle.h

```

#ifndef VEHICLE_H
#define VEHICLE_H

#include
#include
using namespace std;

class Vehicle {
protected:
    string brand;
    int year;

public:
    Vehicle(string b, int y); // Constructor

```

```

        // Common method for all vehicles
        virtual void displayInfo() const;

        virtual ~Vehicle(); // Virtual destructor
};

#endif
Vehicle.cpp

#include "Vehicle.h"

Vehicle::Vehicle(string b, int y) : brand(b), year(y) {}

void Vehicle::displayInfo() const {
    cout << "Brand: " << brand << ", Year: " << year << endl;
}

Vehicle::~~Vehicle() {}
Car.h

#ifndef CAR_H
#define CAR_H

#include "Vehicle.h"

class Car : public Vehicle {
private:
    int doors;

public:
    Car(string b, int y, int d); // Constructor

    // Overridden method for Car
    void displayInfo() const override;
};

#endif
Car.cpp

#include "Car.h"

Car::Car(string b, int y, int d) : Vehicle(b, y), doors(d) {}

void Car::displayInfo() const {
    Vehicle::displayInfo(); // Call base class method
    cout << "Doors: " << doors << endl;
}
Truck.h

#ifndef TRUCK_H

```



```

#define TRUCK_H

#include "Vehicle.h"

class Truck : public Vehicle {
private:
    double loadCapacity;

public:
    Truck(string b, int y, double capacity); // Constructor

    // Overridden method for Truck
    void displayInfo() const override;
};

#endif
Truck.cpp

#include "Truck.h"

Truck::Truck(string b, int y, double capacity) : Vehicle(b, y),
loadCapacity(capacity) {}

void Truck::displayInfo() const {
    Vehicle::displayInfo(); // Call base class method
    cout << "Load Capacity: " << loadCapacity << " tons" << endl;
}
Main.cpp

#include "Car.h"
#include "Truck.h"

int main() {
    Vehicle* vehicles[2];

    vehicles[0] = new Car("Toyota", 2020, 4);
    vehicles[1] = new Truck("Ford", 2018, 3.5);

    for (int i = 0; i < 2; i++) {
        vehicles[i]->displayInfo(); // Polymorphism: calls the
correct displayInfo method
    }

    // Cleanup
    delete vehicles[0];
    delete vehicles[1];
}

```

3 Operator Overloading: Customizing Operators

Operator overloading allows you to redefine the behavior of operators for user-defined types. For example, you can overload + to add two objects of a custom class together.

Code Example:

ComplexNumber.h

```
#ifndef COMPLEXNUMBER_H
#define COMPLEXNUMBER_H

#include
using namespace std;

class ComplexNumber {
private:
    double real, imag;

public:
    ComplexNumber(double r = 0, double i = 0); // Constructor

    // Overload '+' operator to add two complex numbers
    ComplexNumber operator+(const ComplexNumber& other) const;

    // Method to display complex number
    void display() const;
};
```

#endif

ComplexNumber.cpp

```
#include "ComplexNumber.h"

ComplexNumber::ComplexNumber(double r, double i) : real(r),
imag(i) {}

ComplexNumber ComplexNumber::operator+(const ComplexNumber& other)
const {
    return ComplexNumber(real + other.real, imag + other.imag);
}

void ComplexNumber::display() const {
    cout << real << " + " << imag << "i" << endl;
}
```

Main.cpp

```
#include "ComplexNumber.h"

int main() {
    ComplexNumber c1(1.5, 2.5);
    ComplexNumber c2(3.5, 4.5);

    ComplexNumber c3 = c1 + c2; // Using overloaded '+' operator
    c3.display();
}
```

4 Abstract Classes and Interfaces: Defining Common Behavior

An **abstract class** cannot be instantiated and usually contains pure virtual functions. It defines a common interface that derived classes must implement. This is useful when you want to ensure that certain methods are always implemented in derived classes.

Code Example:

PaymentMethod.h

```
#ifndef PAYMENTMETHOD_H
#define PAYMENTMETHOD_H

class PaymentMethod {
public:
    virtual void processPayment(double amount) = 0; // Pure
virtual function
    virtual ~PaymentMethod() {}
};

#endif
```

CreditCardPayment.h

```
#ifndef CREDITCARDPAYMENT_H
#define CREDITCARDPAYMENT_H

#include "PaymentMethod.h"
#include <string>
using namespace std;

class CreditCardPayment : public PaymentMethod {
private:
    string cardNumber;

public:
    CreditCardPayment(string cardNum); // Constructor
    void processPayment(double amount) override; // Method
implementation
};

#endif
```

CreditCardPayment.cpp

```
#include "CreditCardPayment.h"
#include <iostream>

CreditCardPayment::CreditCardPayment(string cardNum) :
cardNumber(cardNum) {}

void CreditCardPayment::processPayment(double amount) {
```

```

        cout << "Processing Credit Card payment of " << amount << "
for card " << cardNumber << endl;
}

```

Main.cpp

```

#include "CreditCardPayment.h"
#include
using namespace std;

int main() {
    PaymentMethod* payment = new
CreditCardPayment("1234-5678-9876");
    payment->processPayment(100.0); // Calls the 'processPayment'
method of CreditCardPayment
    delete payment;
}

```

5 Composition vs. Inheritance: Including Objects vs. Extending Classes

Composition involves including objects as members of another class, while **inheritance** allows one class to extend another. Composition is often considered a "has-a" relationship, while inheritance is a "is-a" relationship.

Code Example:

Person.h

```

#ifndef PERSON_H
#define PERSON_H

#include <string>
using namespace std;

class Person {
private:
    string name;

public:
    Person(string n); // Constructor
    virtual void displayInfo() const; // Virtual function for
polymorphism
    virtual ~Person() = default;
};

```

#endif

Person.cpp

```

#include "Person.h"
#include
using namespace std;

Person::Person(string n) : name(n) {}

```

```
void Person::displayInfo() const {
    cout << "Name: " << name << endl;
}
```

LibraryMember.h

```
#ifndef LIBRARYMEMBER_H
#define LIBRARYMEMBER_H

#include "Person.h"

class LibraryMember : public Person {
private:
    string memberId;

public:
    LibraryMember(string name, string id); // Constructor
    void displayInfo() const override; // Overriding Person's
method
};

#endif
```

LibraryMember.cpp

```
#include "LibraryMember.h"
#include <iostream>

LibraryMember::LibraryMember(string name, string id) :
Person(name), memberId(id) {}

void LibraryMember::displayInfo() const {
    Person::displayInfo(); // Call base class method
    cout << "Member ID: " << memberId << endl;
}
```

Book.h

```
#ifndef BOOK_H
#define BOOK_H

#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;

public:
    Book(string t, string a); // Constructor
    void displayInfo() const; // Method to display book info
};
```

```
#endif
```

Book.cpp

```
#include "Book.h"
#include <iostream>
using namespace std;
```

```
Book::Book(string t, string a) : title(t), author(a) {}
```

```
void Book::displayInfo() const {
    cout << "Title: " << title << ", Author: " << author << endl;
}
```

Library.h

```
#ifndef LIBRARY_H
#define LIBRARY_H
```

```
#include "Book.h"
#include <vector>
using namespace std;
```

```
class Library {
private:
    vector<Book> books; // Composition: Library "has-a" Book

public:
    void addBook(const Book& book); // Method to add books
    void displayBooks() const; // Method to display books
};
```

```
#endif
```

Library.cpp

```
#include "Library.h"
```

```
void Library::addBook(const Book& book) {
    books.push_back(book);
}
```

```
void Library::displayBooks() const {
    for (const auto& book : books) {
        book.displayInfo(); // Call Book's displayInfo
    }
}
```

Main.cpp

```
#include <iostream>
using namespace std;
#include "Person.h"
#include "LibraryMember.h"
```

```

#include "Book.h"
#include "Library.h"

int main()
{
    // Demonstrate polymorphism with Person and LibraryMember
    Person* person1 = new Person("Alice");
    Person* member1 = new LibraryMember("Bob", "M123");
    cout << "=== Person Info ===" << endl;
    person1->displayInfo(); // Calls Person::displayInfo
    cout << "\n=== Library Member Info ===" << endl;
    member1->displayInfo();
    // Calls LibraryMember::displayInfo (overridden)
    // Clean up
    delete person1;
    delete member1;
    // Demonstrate Library and Book
    Library library;
    Book book1("1984", "George Orwell");
    Book book2("To Kill a Mockingbird", "Harper Lee");
    library.addBook(book1);
    library.addBook(book2);
    cout << "\n=== Library Books ===" << endl;
    library.displayBooks();

    return 0;
}

```

6 Dynamic cast (polymorphism)

Use `dynamic_cast` to safely convert a base class pointer to a derived class pointer in a polymorphic class hierarchy (must have at least one virtual function).

Why use it?

To access subclass-specific functionality while ensuring the cast is valid at runtime.

Example:

```

#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void speak() { cout << "Some animal sound\n"; }
};

class Dog : public Animal
{
public:

```

```

    void speak() override { cout << "Woof!\n"; }
    void fetch() { cout << "Dog is fetching a ball.\n"; }
};

void interactWithAnimal(Animal* a)
{
    a->speak();

    // Safe downcast to Dog
    if (Dog* d = dynamic_cast<Dog*>(a))
    {
        d->fetch(); // Only if 'a' is really a Dog
    }
    else
    {
        cout << "This animal is not a dog.\n";
    }
}

int main()
{
    Animal* a1 = new Dog;
    Animal* a2 = new Animal;

    interactWithAnimal(a1); // Output: Woof! Dog is fetching a
ball.
    interactWithAnimal(a2); // Output: Some animal sound. This
animal is not a dog.

    delete a1;
    delete a2;
    return 0;
}

```


File Handling

1 File Handling: Basics of **ifstream**, **ofstream**, and **fstream**

File handling allows programs to read from and write to files. This chapter covers basic file operations using three essential classes:

- **ifstream** – for reading from files.
- **ofstream** – for writing to files.
- **fstream** – for both reading from and writing to files.

Writing Data to a File

The **ofstream** class is used to create files and write to them. When a file is opened with **ofstream**, the content is written at the beginning (overwriting the file). If the file doesn't exist, it is created.

Example: Writing Data to a File (**ofstream**)

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ofstream outFile("example.txt");
    if (outFile.is_open()) {
        outFile << "This is a simple file write example." << endl;
        outFile.close();
    } else {
        cerr << "Error opening file for writing." << endl;
    }
    return 0;
}
```

Reading Data from a File

The **ifstream** class is used to open a file and read from it. It is essential to check if the file has been opened successfully before reading from it.

Example: Reading Data from a File (**ifstream**)

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
```

```

    ifstream inFile("example.txt");
    string line;
    if (inFile.is_open()) {
        while (getline(inFile, line)) {
            cout << line << endl;
        }
        inFile.close();
    } else {
        cerr << "Error opening file for reading." << endl;
    }
    return 0;
}

```

2 Appending Data to a File Using **fstream**

The **fstream** class is used for both input and output operations. You can append to an existing file without overwriting it by opening the file in append mode (**ios::app**).

Example: Appending Data to a File (**fstream**)

```

#include <fstream>
#include <iostream>
using namespace std;

int main() {
    fstream file("example.txt", ios::out | ios::app); // Open file
in append mode
    if (file.is_open()) {
        file << "Appending new line to the file." << endl;
        file.close();
    } else {
        cerr << "Error opening file for appending." << endl;
    }
    return 0;
}

```

3 File Error Management

It is crucial to handle errors while working with files to ensure that the program does not crash or produce unwanted results. C++ provides several ways to handle errors, such as checking if the file stream was successfully opened.

Error Handling during File Writing

```

ofstream outFile("example.txt");
if (!outFile) {
    cerr << "Error: Could not open file for writing." << endl;
    return 1; // Exit if file cannot be opened
}

```

```
}
```

Error Handling during File Reading

```
ifstream inFile("example.txt");
if (!inFile) {
    cerr << "Error: Could not open file for reading." << endl;
    return 1; // Exit if file cannot be opened
}
```

By checking the success of file operations before reading or writing, you can prevent errors from propagating and provide useful feedback to the user.

4 Binary File Manipulation

Binary files allow you to store data in a format that is not human-readable but can be more efficient for saving large amounts of data. C++ supports reading and writing binary data using **fstream** with binary mode (`ios::binary`).

Writing to a Binary File

```
#include <fstream>
using namespace std;

struct Student {
    string name;
    int age;
};

int main() {
    Student s1 = {"John Doe", 21};
    fstream file("student.dat", ios::out | ios::binary);
    if (file.is_open()) {
        file.write(reinterpret_cast<const char*>(&s1),
sizeof(s1)); // Writing binary data
        file.close();
    } else {
        cerr << "Error opening binary file for writing." << endl;
    }
    return 0;
}
```

Reading from a Binary File

```
#include <fstream>
#include <iostream>
using namespace std;

struct Student {
    string name;
    int age;
};

int main() {
```

```

        Student s1;
        fstream file("student.dat", ios::in | ios::binary);
        if (file.is_open()) {
            file.read(reinterpret_cast<char*>(&s1), sizeof(s1)); //
Reading binary data
            cout << "Name: " << s1.name << ", Age: " << s1.age <<
endl;
            file.close();
        } else {
            cerr << "Error opening binary file for reading." << endl;
        }
        return 0;
    }

```

5 Serialization and Deserialization (Saving and Loading Data)

Serialization refers to converting an object into a format that can be easily stored or transmitted (e.g., saving to a file). **Deserialization** is the process of reading the serialized data back into an object.

Example: Serialization & Deserialization

Employee.h

```

#ifndef EMPLOYEE_H
#define EMPLOYEE_H

#include <string>
using namespace std;

class Employee {
private:
    string name;
    int id;
    double salary;

public:
    Employee();
    Employee(const string& name, int id, double salary);

    void serialize(const string& filename) const;
    void deserialize(const string& filename);

    void display() const;
};

#endif
Employee.cpp

```

```

#include "Employee.h"

```

```

#include <fstream>
#include <iostream>

Employee::Employee() : name(""), id(0), salary(0.0) {}

Employee::Employee(const string& name, int id, double salary)
    : name(name), id(id), salary(salary) {}

void Employee::serialize(const string& filename) const {
    ofstream outFile(filename, ios::binary);
    if (!outFile) {
        cerr << "Failed to open file for writing." << endl;
        return;
    }

    size_t nameLen = name.size();
    outFile.write(reinterpret_cast<const char*>(&nameLen),
sizeof(nameLen));
    outFile.write(name.c_str(), nameLen);
    outFile.write(reinterpret_cast<const char*>(&id), sizeof(id));
    outFile.write(reinterpret_cast<const char*>(&salary),
sizeof(salary));

    outFile.close();
}

void Employee::deserialize(const string& filename) {
    ifstream inFile(filename, ios::binary);
    if (!inFile) {
        cerr << "Failed to open file for reading." << endl;
        return;
    }

    size_t nameLen;
    inFile.read(reinterpret_cast<char*>(&nameLen),
sizeof(nameLen));
    name.resize(nameLen);
    inFile.read(&name[0], nameLen);
    inFile.read(reinterpret_cast<char*>(&id), sizeof(id));
    inFile.read(reinterpret_cast<char*>(&salary), sizeof(salary));

    inFile.close();
}

void Employee::display() const {
    cout << "Name: " << name << "\nID: " << id << "\nSalary: " <<
salary << endl;
}
main.cpp

#include <iostream>

```

```

#include "Employee.h"
using namespace std;

int main() {
    Employee e1("Alice Johnson", 101, 75000.0);

    // Serialize to file
    e1.serialize("employee.dat");

    // Deserialize into a new object
    Employee loaded;
    loaded.deserialize("employee.dat");

    cout << "Deserialized Employee:\n";
    loaded.display();

    return 0;
}

```

6 File Operations for Managing Student Records

In this example, we implement functions to manage student records, including **writing** to a file, **reading** from it, and **updating** the records.

Saving Student Data to a File

```

void Student::writeToFile(const string& filename) {
    fstream fileToWrite(filename, ios::out | ios::app |
ios::binary);
    if (fileToWrite.is_open()) {
        fileToWrite.write(reinterpret_cast(this), sizeof(*this));
// Writing student data
        fileToWrite.close();
    } else {
        cerr << "Couldn't open the file." << endl;
    }
}

```

Reading Student Data from a File

```

void Student::readFromFile(const string& filename) const {
    fstream fileToRead(filename, ios::in | ios::binary);
    if (fileToRead.is_open()) {
        while (fileToRead.read(reinterpret_cast(this),
sizeof(*this))) {
            cout << "Student Info: " << name << ", " << age << ",
" << gpa << endl;
        }
        fileToRead.close();
    } else {
        cerr << "Couldn't open the file." << endl;
    }
}

```

```
}  
}
```

✓ Summary of Key Concepts

- **Basic File Handling:** Use `ifstream`, `ofstream`, and `fstream` for file reading, writing, and both operations.
- **File Error Management:** Always check for successful file operations to ensure smooth program execution.
- **Binary File Manipulation:** Learn to work with binary files using `fstream` for efficient data storage.
- **Serialization & Deserialization:** Save and load object data in binary format, making it easy to store and retrieve complex data structures.
- **Managing Records:** Use file operations to manage student records, including saving, reading, and updating data.

📖 Standard Template Library (STL)

1 Introduction to the Standard Template Library (STL)

The **Standard Template Library (STL)** in C++ provides several essential features that help improve code efficiency and readability. It includes a collection of **containers**, **algorithms**, and **iterators** to manage collections of data and perform operations on them.

Key components of the STL:

- **Containers:** Store data in various forms.
- **Algorithms:** Perform operations such as searching, sorting, and modifying data.
- **Iterators:** Provide a way to access and manipulate elements in containers.

2 Containers in STL

STL containers store and organize data. Some commonly used containers are:

- **vector**: A dynamic array that grows as needed.
- **list**: A doubly linked list.
- **deque**: A double-ended queue.
- **map / unordered_map**: Key-value pair containers.
- **set / unordered_set**: Stores unique elements.
- **stack**: A container adapter that provides stack-like operations (LIFO).
- **queue**: A container adapter that provides queue-like operations (FIFO).
- **priority_queue**: A container adapter that provides priority queue operations.

3 **vector**: Dynamic Array

The **vector** container in STL is one of the most commonly used containers. It provides dynamic array functionality, where the size of the container can grow or shrink dynamically as needed.

Key Characteristics of **vector:**

- **Dynamic Size**: Unlike arrays, vectors can resize themselves automatically when elements are added or removed.
- **Random Access**: Allows fast random access to elements by index.
- **Efficient Append**: Adding elements to the end of a vector is typically fast.
- **Internal Storage**: Vectors use contiguous memory storage, similar to arrays, but are more flexible.

Use Cases of **vector**:

- When you need a resizable array that provides random access to elements.
- When you want to frequently add elements at the end of the collection.

Example 1: Basic Operations with **vector**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Create a vector of integers and add elements
    vector<int> numbers = {10, 20, 30, 40, 50};

    // Add elements using push_back
    numbers.push_back(60);

    // Print elements
    for (int num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

10 20 30 40 50 60

4 Iterators (STL Navigation)

Iterators are objects used to **traverse containers** (like `vector`, `list`, etc.) in a consistent and generic way. Think of them like pointers that can move through elements.

◆ Key Concepts:

- `begin()` returns an iterator to the first element
- `end()` returns an iterator **one past** the last element
- You can increment (`++`) or decrement (`--`) iterators depending on the container
- Iterators support access (`*it`) and comparison (`==`, `!=`)

◆ Types:

- `iterator` – can read/write
- `const_iterator` – read-only
- `reverse_iterator` – goes backward

✓ Example with **vector**:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<string> fruits = {"apple", "banana", "cherry"};

    // Regular iterator – Modify first element
    vector<string>::iterator it = fruits.begin();
    *it = "grape"; // replaces "apple"

    // const_iterator – Read-only traversal
    cout << "Using const_iterator: ";
    vector<string>::const_iterator cit;
    for (cit = fruits.cbegin(); cit != fruits.cend(); cit++)
    {
        cout << *cit << " ";
    }
}
```

```

    }
    cout << endl;

    // reverse_iterator – Traverse in reverse order
    cout << "Using reverse_iterator: ";
    vector<string>::reverse_iterator rit;
    for (rit = fruits.rbegin(); rit != fruits.rend(); rit++)
    {
        cout << *rit << " ";
    }
    cout << endl;

    return 0;
}

```

Output:

```

Using const_iterator: grape banana cherry
Using reverse_iterator: cherry banana grape

```

5 Working with `list` (STL Linked List)

`list` is a **doubly-linked list** provided by the C++ Standard Template Library. It allows **efficient insertion and deletion** at any position in constant time.

Key Properties:

- Elements are **not contiguous** in memory (unlike `vector`)
- Supports **bidirectional iteration**
- Best used when frequent **insertions/removals** are needed

Common Functions:

- `push_back()` / `push_front()` – Add elements at the back/front
- `pop_back()` / `pop_front()` – Remove elements
- `insert()`, `erase()` – Insert or delete at a specific position
- `sort()`, `remove()`, `clear()` – Useful list operations

Example:

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    list<int> numbers;

    // Add elements
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_front(5);

    // Output: 5 10 20
    for (int n : numbers) {
        cout << n << " ";
    }
    cout << endl;

    // Remove element
    numbers.pop_front(); // Removes 5

    // Insert 15 after first element (iterator)
    auto it = numbers.begin();
    ++it; // move to second element
    numbers.insert(it, 15); // Insert before 20

    // Output: 10 15 20
    for (int n : numbers) {
        cout << n << " ";
    }
    cout << endl;

    // Sort (not needed here but available)
    numbers.sort();

    return 0;
}
```

Output:

```
5 10 20
10 15 20
```

When to Use `list`?

-  Efficient insert/delete in the **middle** of the list
-  Slower than **vector** for **random access** or simple appends

6 **stack**: Last In, First Out (LIFO) Structure

The **stack** is a container adapter that gives you a LIFO (Last In, First Out) structure, meaning elements are added and removed from the same end, known as the top of the stack.

Example: Using **stack**

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s;

    // Pushing elements onto the stack
    s.push(10); // Container state: [10]
    s.push(20); // Container state: [10, 20]
    s.push(30); // Container state: [10, 20, 30]

    // Popping elements from the stack
    cout << "First pop: ";
    s.pop(); // First pop: Container state: [10, 20]

    cout << "Second pop: ";
    s.pop(); // Second pop: Container state: [10]

    cout << "Third pop: ";
    s.pop(); // Third pop: Container state: [empty]

    cout << endl;

    return 0;
}
```

7 **queue**: First In, First Out (FIFO) Structure

The **queue** is a container adapter that follows FIFO (First In, First Out) ordering. It is used when you need to manage items in a sequence where the first item inserted is the first to be removed.

Example: Using **queue**

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    // Pushing elements into the queue
```

```

q.push(10); // Container state: [10]
q.push(20); // Container state: [10, 20]
q.push(30); // Container state: [10, 20, 30]

// Popping elements from the queue
cout << "First pop: ";
q.pop(); // First pop: Container state: [20, 30]

cout << "Second pop: ";
q.pop(); // Second pop: Container state: [30]

cout << "Third pop: ";
q.pop(); // Third pop: Container state: [empty]

cout << endl;

return 0;
}

```

8 **priority_queue**: A Priority Queue

The **priority_queue** is a container adapter that gives access to the largest (or smallest) element. The default behavior creates a max-heap, where the largest element is always at the top. However, you can customize it to act as a min-heap.

Example: Using **priority_queue**

```

#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;

    // Inserting elements into the priority queue
    pq.push(10); // Container state: [10]
    pq.push(30); // Container state: [30, 10]
    pq.push(20); // Container state: [30, 20, 10]

    // Displaying and removing elements from the priority queue
    cout << "First pop: ";
    pq.pop(); // First pop: Container state: [20, 10]

    cout << "Second pop: ";
    pq.pop(); // Second pop: Container state: [10]

    cout << "Third pop: ";
    pq.pop(); // Third pop: Container state: [empty]
}

```

```

    cout << endl;

    return 0;
}

```

9 unordered_map: Hash-Based Key-Value Pair Storage

The **unordered_map** is a hash-based container that stores key-value pairs. It provides faster access to elements using a key but does not maintain any specific order of elements. It is generally more efficient than **map** in terms of lookup time due to its underlying hash table structure.

Example: Using unordered_map

```

#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<string, int> populationMap;

    // Adding key-value pairs to the unordered_map
    populationMap["USA"] = 331002651;
    populationMap["India"] = 1380004385;
    populationMap["China"] = 1439323776;

    // Accessing elements by key
    cout << "Population of India: " << populationMap["India"] <<
endl;

    return 0;
}

```

Output:

Population of India: 1380004385

Container state after the operation:

```
{ "USA" : 331002651, "India" : 1380004385, "China" : 1439323776 }
```

10 Lambda Expressions (C++11 and Beyond)

Lambdas are **anonymous functions** you can define inline. They're useful for short operations passed to algorithms, custom sorting, filtering, or quick function-like behavior.

◆ Syntax:

```
[capture](parameters) -> return_type { body }
```

- []: Capture variables (by value [=], by reference [&], or specific [x, &y])
- Parameters and return types are optional (can be inferred)

✓ Example:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> nums = {3, 7, 2, 9, 4};

    // Lambda to print elements
    auto print = [](int x) { cout << x << " "; };

    cout << "Numbers: ";
    for_each(nums.begin(), nums.end(), print);
    cout << endl;

    // Lambda to check if a number is even
    auto isEven = [](int x) { return x % 2 == 0; };

    cout << "Even numbers: ";
    for (int n : nums)
    {
        if (isEven(n))
        {
            cout << n << " ";
        }
    }

    return 0;
}
```

Output:

```
Numbers: 3 7 2 9 4
Even numbers: 2 4
```


11 STL Algorithm + Iterators + Lambda Expressions

You can combine **STL algorithms**, **iterators**, and **lambda expressions** for clean and powerful operations on containers.

✓ Example:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> data = {4, 7, 1, 9, 2, 6};

    // Sort in descending order using lambda
    sort(data.begin(), data.end(), [](int a, int b) { return a >
b;});

    // Find first element greater than 5
    auto it = find_if(data.begin(), data.end(), [](int x) { return
x > 5;});

    // Print sorted data
    cout << "Sorted (desc): ";
    for (int x : data)
    {
        cout << x << " ";
    }
    cout << endl;

    // Print found value
    if (it != data.end())
    {
        cout << "First value > 5: " << *it << endl;
    }
    else
    {
        cout << "No value > 5 found." << endl;
    }

    return 0;
}
```

Output:

```
Sorted (desc): 9 7 6 4 2 1
First value > 5: 9
```

✓ Summary of Key Concepts

- **Containers:**
 - `list`: A doubly linked list.
 - `stack`: LIFO (Last In, First Out) structure.
 - `queue`: FIFO (First In, First Out) structure.
 - `priority_queue`: Access the largest (or smallest) element first.
 - `unordered_map`: Hash-based key-value pair container for fast access.
 - `map`: Sorted key-value pair container.
 - `set`: Stores unique elements in sorted order.
 - `vector`: Dynamic array with random access and efficient append operations.
- **Iterators**: Traverse through containers and access elements efficiently.
- **Lambda Expressions**: Use inline functions to simplify operations like filtering and transforming data.
- **STL Algorithms**: Apply standard algorithms like `sort`, `find`, and `count` to manipulate and query containers.
- **Container Adapters**: Understand the usage of `stack`, `queue`, and `priority_queue` for specific data structure needs.

Generic Programming and Templates

1 Introduction to Generic Programming and Templates

Generic programming in C++ allows writing functions and classes that can operate with any data type. Templates are the core feature of generic programming. They allow you to define functions, classes, and data structures that work with any type without needing to rewrite the code for each type.

In C++, templates enable you to write type-independent code that is still type-safe. With templates, C++ gives you the ability to work with **types** as **parameters**.

2 Function Templates: Defining Functions for Any Type

A **function template** is a function that can work with any data type. Instead of defining multiple functions for each type, you write one function with a **template parameter** that can be used for any data type.

Code Example: Swapping Two Variables

```
#include <iostream>

template <typename T>
void swapValues(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swapValues(x, y);
    std::cout << "x: " << x << ", y: " << y << std::endl;

    double a = 1.5, b = 2.5;
    swapValues(a, b);
    std::cout << "a: " << a << ", b: " << b << std::endl;

    return 0;
}
```

Output:

```
x: 20, y: 10
a: 2.5, b: 1.5
```

3 Class Templates: Defining Classes for Any Type

A **class template** allows you to define a class that works with any type. By using templates, a single class definition can be used for different data types.

Code Example: A Generic Pair Class

```
#include <iostream>

template <typename T>
class Pair {
private:
    T first;
    T second;

public:
    Pair(T f, T s) : first(f), second(s) {}

    void display() {
        std::cout << "First: " << first << ", Second: " << second
<< std::endl;
    }
};

int main() {
    Pair<int> intPair(1, 2);
    intPair.display();

    Pair<std::string> stringPair("Hello", "World");
    stringPair.display();

    return 0;
}
```

Output:

```
First: 1, Second: 2
First: Hello, Second: World
```

4 Operator Overloading with Class Templates

You can **overload operators** for class templates, allowing the use of operators like +, -, *, etc., with user-defined types.

Code Example: Complex Number Class with Operator Overloading

```
#include <iostream>

template <typename T>
class ComplexNumber {
```

```

private:
    T real;
    T imag;

public:
    ComplexNumber(T r, T i) : real(r), imag(i) {}

    // Overload the '+' operator
    ComplexNumber<T> operator+(const ComplexNumber<T>& other) {
        return ComplexNumber<T>(real + other.real, imag +
other.imag);
    }

    void display() const {
        std::cout << real << " + " << imag << "i" << std::endl;
    }
};

int main() {
    ComplexNumber<int> c1(1, 2), c2(3, 4);
    ComplexNumber<int> c3 = c1 + c2;

    c3.display(); // Output: 4 + 6i

    return 0;
}

```

Output:

4 + 6i

5 Template Specialization: Customizing Behavior for Specific Types

Template specialization allows you to define custom behavior for a specific data type while maintaining the general template for other types.

Code Example: Specialized Print Function

```

#include <iostream>

template <typename T>
void print(const T& value) {
    std::cout << value << std::endl;
}

// Specialized version for const char*
template <>
void print<const char*>(const char*& value) {
    std::cout << "C-string: " << value << std::endl;
}

```

```
int main() {
    print(42);           // Regular template
    print(3.14);         // Regular template
    print("Hello World"); // Specialized template

    return 0;
}
```

Output:

```
42
3.14
C-string: Hello World
```

6 Template Metaprogramming: Calculating Values at Compile-Time

Template metaprogramming enables you to perform computations at compile-time, often used to optimize code or generate types.

Code Example: Compile-Time Greatest Common Divisor (GCD)

Let's calculate the **greatest common divisor (GCD)** at compile-time using template recursion.

```
template <int A, int B>
struct GCD {
    static const int value = GCD<B, A % B>::value;
};

template <int A>
struct GCD<A, 0> {
    static const int value = A;
};

int main() {
    std::cout << "GCD of 56 and 98: " << GCD<56, 98>::value <<
std::endl; // Output: 14
    return 0;
}
```

Output:

```
GCD of 56 and 98: 14
```

7 Concept-Based Programming (C++20)

C++20 introduced **concepts** to constrain the types that can be passed to template parameters. This allows for better compile-time checking of types.

Code Example: Numeric Concept for Arithmetic Operations

This example demonstrates a concept that ensures that only numeric types can be used in the `add` function.

```
#include <iostream>

template<typename T>
concept Numeric = std::is_arithmetic<T>::value;

template<Numeric T>
T add(T a, T b) {
    return a + b;
}

int main() {
    std::cout << add(5, 3) << std::endl;          // Works fine
    std::cout << add(3.14, 2.71) << std::endl;    // Works fine

    // Uncommenting the following line would cause a compile-time
    error:
    // std::cout << add("Hello", "World") << std::endl; //
    Compile-time error

    return 0;
}
```

Output:

```
8
5.85
```

Memory Management and Optimization

1 Introduction to Memory Management and Optimization

Memory management in C++ involves controlling how memory is allocated and deallocated, ensuring that resources are efficiently used and no memory is leaked. Optimization, on the other hand, focuses on improving the performance of the program through efficient code and algorithm choices.

In C++, memory can be managed both manually using `new/delete` or automatically with modern tools like smart pointers. Optimizing memory and code can dramatically increase a program's speed and reduce its resource usage.

2 Dynamic Memory Management with `new` and `delete`

Dynamic memory in C++ is allocated using the `new` keyword and deallocated using `delete`. This allows memory to be allocated at runtime, providing flexibility but requiring careful management to avoid memory leaks.

Code Example: Dynamic Array for Storing Integers

```
#include <iostream>
using namespace std;

int* createArray(int size) {
    return new int[size]; // dynamically allocate array
}

void deleteArray(int* array) {
    delete[] array; // deallocate array memory
}

int main() {
    int* array = createArray(5); // allocate array of 5 integers
    for (int i = 0; i < 5; ++i) {
        array[i] = i * 10; // assigning values
    }

    for (int i = 0; i < 5; ++i) {
        cout << array[i] << " "; // output values
    }
    cout << endl;

    deleteArray(array); // free memory
    return 0;
}
```



```
}
```

Output:

```
0 10 20 30 40
```

Explanation:

In this example, the `createArray` function allocates memory for an integer array at runtime, and the `deleteArray` function frees the memory once it's no longer needed.

3

Smart Resource Management with Smart Pointers

Smart pointers are an advanced feature in C++ that provide automatic memory management. They are safer than raw pointers because they automatically deallocate memory when it's no longer in use, preventing memory leaks.

Code Example: Using `unique_ptr` for Memory Management

```
#include <iostream>
#include <memory>
using namespace std;

class Person {
public:
    Person(const string& name) : name(name) {}
    void greet() const { cout << "Hello, " << name << "!" << endl; }
private:
    string name;
};

int main() {
    unique_ptr<Person> personPtr = make_unique<Person>("John");
    personPtr->greet();
    // No need to call delete; the memory is automatically managed
    return 0;
}
```

Output:

```
Hello, John!
```

Explanation:

In this example, `std::unique_ptr` automatically manages the memory for the `Person` object. The memory is released as soon as the pointer goes out of scope, preventing memory leaks.

4 Memory Optimization with **inline** and **constexpr**

Memory optimization can also be achieved by using **inline** functions and **constexpr** functions, which reduce the overhead of function calls and enable compile-time calculations.

Code Example: Optimized Calculation of Area

```
#include <iostream>
using namespace std;

constexpr double calculateArea(double radius) {
    return 3.14159 * radius * radius; // compile-time calculation
}

inline int add(int a, int b) {
    return a + b; // eliminates function call overhead
}

int main() {
    double area = calculateArea(5.0); // computed at compile-time
    cout << "Area: " << area << endl;

    int sum = add(10, 20); // no overhead from function call
    cout << "Sum: " << sum << endl;

    return 0;
}
```

Output:

```
Area: 78.53975
Sum: 30
```

Explanation:

Here, `calculateArea` is marked as `constexpr`, allowing it to be evaluated at compile time, which improves performance. The `add` function is marked as `inline`, reducing function call overhead.

5 Memory Management with Custom Allocators

Custom allocators give you fine control over memory allocation and deallocation, which can be useful in performance-critical applications.

Code Example: Simple Custom Allocator

```
#include <iostream>
#include <cstdlib>
using namespace std;

class CustomAllocator {
public:
    static void* allocate(size_t size) {
        return operator new(size); // allocate memory manually
    }

    static void deallocate(void* pointer) {
        operator delete(pointer); // manually free memory
    }
};

int main() {
    int* ptr =
static_cast<int*>(CustomAllocator::allocate(sizeof(int)));
    *ptr = 100; // assign a value
    cout << "Allocated value: " << *ptr << endl;

    CustomAllocator::deallocate(ptr); // free the memory
    return 0;
}
```

Output:

Allocated value: 100

Explanation:

This code demonstrates a custom memory allocator with the ability to allocate and deallocate memory manually. It uses `std::malloc` and `std::free` for dynamic memory management.

6 Optimizing Memory Usage with Object Pooling

Object pooling is a design pattern that reuses objects from a pool rather than creating new ones, which can be more efficient in scenarios where many objects are frequently created and destroyed.

Code Example: Basic Object Pool for Reusing Objects

```
#include <iostream>
#include <queue>
using namespace std;

class Widget {
public:
    Widget() { cout << "Widget created" << endl; }
    ~Widget() { cout << "Widget destroyed" << endl; }
    void use() { cout << "Using widget" << endl; }
};

class WidgetPool {
private:
    queue<Widget*> pool;

public:
    Widget* acquire() {
        if (pool.empty()) {
            return new Widget(); // create a new object if pool
is empty
        } else {
            Widget* widget = pool.front();
            pool.pop();
            return widget;
        }
    }

    void release(Widget* widget) {
        pool.push(widget); // return object to the pool
    }
};

int main() {
    WidgetPool pool;
    Widget* widget1 = pool.acquire();
    widget1->use();
    pool.release(widget1);

    Widget* widget2 = pool.acquire();
    widget2->use();
    pool.release(widget2);

    return 0;
}
```

Output:

```
Widget created
Using widget
Widget destroyed
Widget created
Using widget
Widget destroyed
```

Explanation:

The object pool reduces the overhead of frequently creating and destroying objects by reusing them. In this example, a pool of `Widget` objects is used to efficiently manage memory.

**Summary of Key Concepts**

- **Dynamic Memory Management:** Allocate and deallocate memory at runtime using `new` and `delete`.
- **Smart Pointers:** Manage memory automatically with `unique_ptr`, `shared_ptr`, and `weak_ptr`.
- **Optimization Techniques:** Use `inline` and `constexpr` for performance improvements.
- **Memory Pooling:** Reuse objects from a pool to reduce the overhead of frequent object creation and destruction.

Multithreading and Concurrent Programming

1 Introduction to Multithreading and Concurrent Programming

Multithreading allows a program to run multiple threads concurrently, enabling tasks to be executed in parallel. This is particularly useful for applications that perform independent tasks, such as data processing, network communication, or interactive applications. Concurrent programming also involves handling synchronization to avoid conflicts when multiple threads access shared data.

In C++, the `<thread>` library and `std::thread` are used to implement multithreading. Additionally, synchronization techniques such as mutexes and condition variables help in managing shared resources among threads.

2 Creating and Managing Threads in C++

In C++, threads are created using the `std::thread` class. This class allows you to run functions in parallel by launching threads in your program.

Code Example: Parallel Calculation of Square Numbers

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>
using namespace std;

mutex coutMutex;

void calculateSquare(int num)
{
    int square = num * num;

    lock_guard<mutex> guard(coutMutex);
    cout << "Square of " << num << " is " << square << endl;
}

int main()
{
    vector<thread> threads;
    for (int i = 1; i <= 5; ++i)
    {
        threads.push_back(thread(calculateSquare, i));
    }

    for (auto& th : threads)
    {
        th.join(); // Wait for all threads to finish
    }

    return 0;
}
```

```
}
```

Output:

```
Square of 1 is 1
Square of 2 is 4
Square of 3 is 9
Square of 4 is 16
Square of 5 is 25
```

Explanation:

This example creates multiple threads to calculate the squares of numbers concurrently. Each thread runs the `calculateSquare` function, and `join()` is used to ensure the main thread waits for all threads to finish.

3**Synchronization with Mutex**

In multithreaded programs, it's crucial to manage access to shared data to prevent data corruption. Mutexes are used to ensure that only one thread can access a resource at a time.

Code Example: Counter with Mutex Synchronization

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx; // Mutex to synchronize access to counter

int counter = 0;

void incrementCounter() {
    for (int i = 0; i < 1000; ++i) {
        std::lock_guard<std::mutex> lock(mtx); // Locking the
mutex
        ++counter;
    }
}

int main() {
    std::thread t1(incrementCounter);
    std::thread t2(incrementCounter);

    t1.join();
    t2.join();

    std::cout << "Final counter value: " << counter << std::endl;
    return 0;
}
```

Output:

```
Final counter value: 2000
```

Explanation:

In this example, two threads increment a shared counter. The mutex `mtx` ensures that only one thread can increment the counter at a time, preventing race conditions and ensuring correct results.

4 Condition Variables for Thread Communication

Condition variables are used to synchronize threads in situations where one thread needs to wait for a condition to be met by another thread.

Code Example: Simple Wait-Notify Mechanism

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>

std::mutex mtx;
std::condition_variable cv;
bool dataReady = false;

void waitForData() {
    std::cout << "Thread waiting for data..." << std::endl;
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, []{ return dataReady; }); // Wait until
dataReady is true
    std::cout << "Data is ready!" << std::endl;
}

void sendData() {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    std::cout << "Data is ready, notifying..." << std::endl;
    {
        std::lock_guard<std::mutex> lock(mtx);
        dataReady = true;
    }
    cv.notify_one(); // Notify one waiting thread
}

int main() {
    std::thread t1(waitForData);
    std::thread t2(sendData);

    t1.join();
    t2.join();

    return 0;
}
```

Output:

```
Thread waiting for data...
Data is ready, notifying...
Data is ready!
```


Explanation:

This example demonstrates the producer-consumer problem, where the first thread waits for the condition `dataReady` to be true, and the second thread sets this condition after a delay. The `cv.notify_one()` call wakes up the waiting thread.

5 Thread Pool for Managing Concurrent Tasks

A thread pool is a collection of reusable threads that can execute multiple tasks. Using a thread pool avoids the overhead of creating and destroying threads for each task.

Code Example: Thread Pool Implementation

```
#include <iostream>
#include <vector>
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

public:
    ThreadPool(size_t numThreads) {
        for (size_t i = 0; i < numThreads; ++i) {
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(this-
>mtx);
                        this->cv.wait(lock, [this] { return !this-
>tasks.empty() || this->stop; });
                        if (this->stop && this->tasks.empty())
                            return;
                        task = std::move(this->tasks.front());
                        this->tasks.pop();
                    }
                    task();
                }
            });
        }

        void enqueue(std::function<void()> task) {
```

```

        {
            std::lock_guard<std::mutex> lock(mtx);
            tasks.push(task);
        }
        cv.notify_one();
    }

    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            stop = true;
        }
        cv.notify_all();
        for (std::thread& worker : workers) {
            if (worker.joinable()) worker.join();
        }
    }
};

int main() {
    ThreadPool pool(3);

    for (int i = 0; i < 10; ++i) {
        pool.enqueue([i] {
            std::cout << "Task " << i << " is being processed." <<
std::endl;
        });
    }

    return 0;
}

```

Output:

```

Task 0 is being processed.
Task 1 is being processed.
Task 2 is being processed.
Task 3 is being processed.
Task 4 is being processed.
Task 5 is being processed.
Task 6 is being processed.
Task 7 is being processed.
Task 8 is being processed.
Task 9 is being processed.

```

Explanation:

This code creates a thread pool with 3 threads. The pool processes 10 tasks concurrently by assigning them to the threads, making the task management more efficient.

6 Asynchronous Programming with `std::future` and `std::promise`

C++11 introduced powerful tools for asynchronous programming, making it easier to run tasks in parallel without managing threads directly. Two key tools are:

- **`std::async`**: Launches a function asynchronously and returns a `std::future` that holds the result.
- **`std::promise`**: Provides an explicit way to pass a result from one thread to another. A `std::future` is used to retrieve the result from the thread that owns the `promise`.

Together, these features enable clear and efficient communication between threads.

Code Example: Running Functions Asynchronously with `std::async`

```
#include <iostream>
#include <thread>
#include <future>
#include <chrono>

void computeSquare(int num, std::promise<int> resultPromise) {
    std::this_thread::sleep_for(std::chrono::seconds(1)); //
    Simulate work
    int result = num * num;
    resultPromise.set_value(result); // Send result to future
}

int main() {
    std::promise<int> promise;
    std::future<int> result = promise.get_future(); // Receive
    result from another thread

    std::thread worker(computeSquare, 10, std::move(promise));

    std::cout << "Performing other tasks while waiting for
    result..." << std::endl;

    int square = result.get(); // Blocks until the value is
    available
    std::cout << "Square of 10 is: " << square << std::endl;

    worker.join(); // Ensure thread completes

    return 0;
}
```

Output:

```
Performing other tasks while waiting for result...
Square of 10 is: 100
```

Explanation:

Here, `std::async` runs the `computeSquare` function asynchronously. The main thread continues execution while waiting for the result, and `result.get()` blocks when the value is needed.

Summary of Key Concepts

- **Threads:** Use `std::thread` to execute functions concurrently.
- **Synchronization:** Employ mutexes and condition variables to manage access to shared resources.
- **Thread Pools:** Use a thread pool to efficiently manage concurrent tasks.
- **Asynchronous Programming:** Use `std::future` and `std::promise` to run functions asynchronously, simplifying concurrency.

1 Introduction to Design Patterns

Design patterns are proven solutions to common problems in software design. They provide a template for writing code that is flexible, reusable, and maintainable. Some commonly used design patterns in C++ include Singleton, Factory, Strategy, Observer, and Command.

In this chapter, we explore the usage of design patterns in solving real-world problems and enhancing the scalability and maintainability of applications. We'll look at how these patterns can be applied to various systems such as notification systems, simulation engines, and more.

2 Singleton Design Pattern: Ensuring a Single Instance

The Singleton pattern restricts the instantiation of a class to one object. This pattern is useful when you need to control access to a shared resource, such as a database connection or a configuration manager.

Code Example: Database Connection Singleton

```
#include <iostream>
#include <mutex>

class DatabaseConnection {
private:
    static DatabaseConnection* instance;
    static std::mutex mtx;

    DatabaseConnection() {
        std::cout << "Database connection established." <<
std::endl;
    }

public:
    static DatabaseConnection* getInstance() {
        std::lock_guard lock(mtx);
        if (instance == nullptr) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    void query(const std::string& sql) {
        std::cout << "Executing query: " << sql << std::endl;
    }
};
```

```

    }
};

DatabaseConnection* DatabaseConnection::instance = nullptr;
std::mutex DatabaseConnection::mtx;

int main() {
    DatabaseConnection* db = DatabaseConnection::getInstance();
    db->query("SELECT * FROM users");

    return 0;
}

```

Output:

Database connection established.
 Executing query: SELECT * FROM users

Explanation:

In this example, DatabaseConnection ensures that only one instance is created using the Singleton pattern. The getInstance() method is used to access the single instance. A mutex is used to ensure thread safety when creating the instance.

3 Factory Design Pattern: Creating Objects

The Factory pattern is used to create objects without specifying the exact class of object that will be created. It allows a class to delegate the responsibility of instantiating objects to a factory class.

Code Example: Car Factory

```

#include <iostream>
#include <string>

class Car {
public:
    virtual void display() const = 0;
};

class Sedan : public Car {
public:
    void display() const override {
        std::cout << "This is a Sedan." << std::endl;
    }
};

class SUV : public Car {
public:
    void display() const override {
        std::cout << "This is an SUV." << std::endl;
    }
}

```

```
};

class CarFactory {
public:
    static Car* createCar(const std::string& type) {
        if (type == "Sedan") {
            return new Sedan();
        } else if (type == "SUV") {
            return new SUV();
        } else {
            return nullptr;
        }
    }
};

int main() {
    Car* myCar = CarFactory::createCar("Sedan");
    myCar->display();
    delete myCar;

    myCar = CarFactory::createCar("SUV");
    myCar->display();
    delete myCar;

    return 0;
}
```

Output:

This is a Sedan.
This is an SUV.

Explanation:

Here, the `CarFactory` creates instances of different car types (Sedan or SUV) based on the provided type. The client does not need to know the specific class of the car; it simply uses the factory to create an instance.

4 Strategy Design Pattern: Defining a Family of Algorithms

The Strategy pattern allows a class to select an algorithm at runtime. It defines a family of algorithms and allows clients to choose one. This is useful when you have multiple ways to perform a task and want to make it interchangeable.

Code Example: Payment Strategy

```
#include <iostream>
#include <memory>

class PaymentStrategy {
```

```

public:
    virtual void pay(int amount) = 0;
};

class CreditCardPayment : public PaymentStrategy {
public:
    void pay(int amount) override {
        std::cout << "Paid " << amount << " using Credit Card." <<
std::endl;
    }
};

class PayPalPayment : public PaymentStrategy {
public:
    void pay(int amount) override {
        std::cout << "Paid " << amount << " using PayPal." <<
std::endl;
    }
};

class PaymentContext {
private:
    std::shared_ptr strategy;

public:
    PaymentContext(std::shared_ptr initialStrategy) :
strategy(initialStrategy) {}

    void setPaymentStrategy(std::shared_ptr newStrategy) {
        strategy = newStrategy;
    }

    void executePayment(int amount) {
        strategy->pay(amount);
    }
};

int main() {
    PaymentContext paymentContext(std::make_shared());
    paymentContext.executePayment(100);

    paymentContext.setPaymentStrategy(std::make_shared());
    paymentContext.executePayment(200);

    return 0;
}

```

Output:

```

Paid 100 using Credit Card.
Paid 200 using PayPal.

```

Explanation:

This example demonstrates the Strategy pattern where different payment methods (Credit Card and PayPal) are encapsulated in separate classes. The `PaymentContext` class allows switching between strategies dynamically.

5 Observer Design Pattern: Notifying Clients of Changes

The Observer pattern allows a subject to notify its observers when a change occurs. This is useful when you have one-to-many relationships, such as when multiple components need to be updated when a single event happens.

Code Example: Weather Station

```
#include <iostream>
#include <vector>
#include <algorithm>

class Observer {
public:
    virtual void update(float temperature) = 0;
    virtual ~Observer() = default;
};

class WeatherStation {
private:
    float temperature;
    std::vector<Observer*> observers;

public:
    void addObserver(Observer* observer) {
        observers.push_back(observer);
    }

    void removeObserver(Observer* observer) {
        observers.erase(std::remove(observers.begin(),
observers.end(), observer), observers.end());
    }

    void setTemperature(float newTemperature) {
        temperature = newTemperature;
        notifyObservers();
    }

    void notifyObservers() {
        for (Observer* observer : observers) {
            observer->update(temperature);
        }
    }
};
```

```

class Display : public Observer {
public:
    void update(float temperature) override {
        std::cout << "Temperature updated to: " << temperature <<
"°C" << std::endl;
    }
};

int main() {
    WeatherStation station;
    Display display;

    station.addObserver(&display);

    station.setTemperature(25.5f);
    station.setTemperature(30.0f);

    return 0;
}

```

Output:

Temperature updated to: 25.5°C
Temperature updated to: 30.0°C

Explanation:

In this example, the `WeatherStation` notifies its observers (in this case, a `Display` object) whenever the temperature is updated. The observers are automatically notified of changes.

Summary of Key Concepts

- **Singleton:** Ensure that a class has only one instance and provide global access to it.
- **Factory:** Create objects without specifying the exact class to instantiate.
- **Strategy:** Define a family of algorithms and make them interchangeable.
- **Observer:** Notify a group of observers about state changes in a subject.

This guide explains the use of advanced design patterns in C++ with original examples. Understanding these patterns can significantly improve the scalability, maintainability, and flexibility of your software systems.